1) When the author talks about pointers, they are not pointers in the modern sense. What are they really?

- ↳ refers to pointers as array indices or subscripts
- ↳ int value that show positions within an array, not memory address (understood today)
- ↳ they would track which element of array is being changed during partitioning & sorting
- ↳ they move through array by incrementing/decrementing int values, pointing to array position rather than holding actual memory address

2) With totally different words, phrasing, and style explain why $N \log N$ is the best possible time for sorting

- ↳ in a tree each comparison split possibilities in groups
- ↳ making a binary decision (item > or < other item), need good enough comparisons to narrow down all possible ordering to 1 correct one.
- ↳ to tell a difference between $N!$ possibilities using binary choice, we need atleast $\log_2(N)!$ comparison
- ↳ using stirling approximation, $\log_2(N)!$ grows proportionally to $N \log N$.
  - ↳ any comparison sorting method makes atleast this many comparison in the worst case, showing that $N \log N$ as the most minimum time

3) The author describes something he calls a "nest." What is a Nest?

- ↳ a saved state of the partition boundaries that need to be sorted
- ↳ in quicksort partitions array into 2 parts, it needs to remember to come back & sort the other partition after done with current one
- ↳ nest stores pairs of indices (left & right boundaries) marking unsorted segments that are waiting to be processed

What is it doing?

- ↳ nest keeps track of pending work, segment of array identified but not yet stored
- ↳ as algorithm work on 1 partition, it pushes the boundaries of other partition onto nest

What would we call it today? → stack

What would we use instead?

- ↳ explicit stack (iterative implementation)
- ↳ call stack through recursion (recursive implimentation), each recursive call saves the state of unfinished partitions

4) Comparing the explanation of the use of nests with the actual implementation, what stands out as strange?

- ↳ the implementation doesn't actually use nest/stack data structure at all
- ↳ described as building up & drawing from a nest of partition boundaries
- ↳ in Algol-60 code, he uses quicksort using goto statements & labels to control the flow, avoiding need for explicit stack structure
- ↳ instead of pushing boundaries onto stack & popping them later, implementation uses program flow control (jumps via goto) to handle partitioning logic
- ↳ var I & J used as working boundaries, & the code jumps between different labeled sections to handle partitioning & moving to next segment
- ↳ this approach is more low level, procedural approach compared to conceptual description of nests, & it avoids overhead of maintaining an explicit stack